



Data Visualization

LABORATORIO

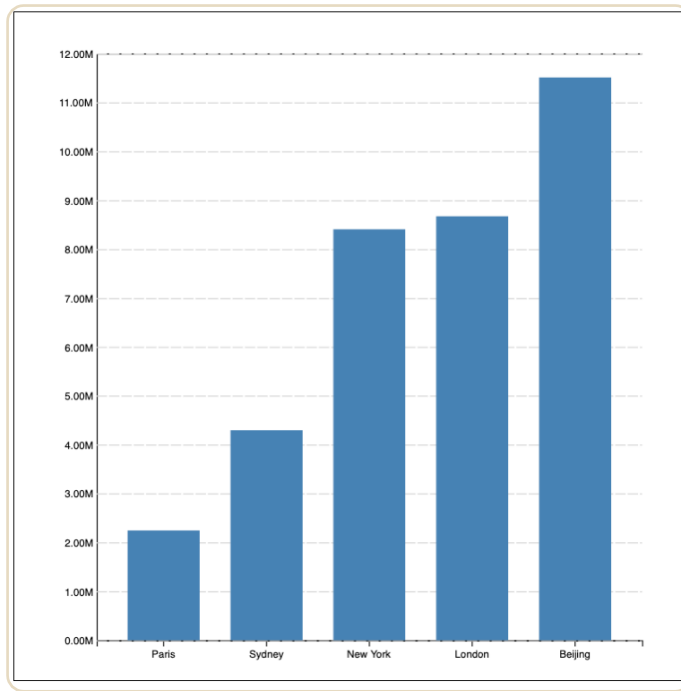
Capitolo 5 – Bar chart



Argomenti del giorno

- Bar chart
 - ScaleBand

Costruzione del Bar Chart



Dati e preprocessing

All'interno della funzione **barchart()** recuperiamo i dati dal csv e trasformiamo i valori della colonna "population" da stringhe a interi

```
async function barchart(){ Show usages new *  
  let data = await getData("./datasets/barchart_data.csv")  
  data = data.map(row => ({  
    ...row,  
    'population': +row['population']  
  }))
```

All'interno della funzione *renderBarChart()* creiamo un gruppo che conterrà il nostro grafico.

Per rendere il grafico più ordinato, possiamo scegliere se ordinare i dati del nostro dataset in maniera crescente tramite la funzione *d3.ascending()*;

```
function renderBarChart(idSVG, data, sort){ Show usages new *  
  
  const svg = d3.select('#'+ idSVG);  
  const svgWidth = svg.attr("width");  
  const svgHeight = svg.attr("height");  
  
  const margin = {top: 40, right: 40, bottom: 40, left: 80};  
  const chartWidth = svgWidth - margin.left - margin.right;  
  const chartHeight = svgHeight - margin.top - margin.bottom;  
  
  const gContainer = svg.append('g')  
    .attr('transform', `translate(${margin.left}, ${margin.top})`)  
  
  if (sort)  
    data.sort((a, b) => d3.ascending(a['population'], b['population']))
```

Definizione scaleX e asse X

Analogamente a quanto visto nelle lezioni precedenti, creiamo la funzione `scaleX` tramite la funzione **`d3.scaleBand()`**: ad ogni città viene assegnata una “**banda**” lungo l’asse X. Il padding serve per lasciare un po’ di spazio tra una barra e l’altra.

Successivamente creiamo l’asse X come visto in precedenza. In questo caso l’asse conterrà i **nomi** delle città sotto la rispettiva barra.

```
//Associamo a ogni città una banda lungo l'asse x  
const scaleX = d3.scaleBand()  
  .domain(data.map(d => d["name"]))  
  .range([0, chartWidth])  
  .padding(0.3);  
  
//Asse X con i nomi delle città  
gContainer.append('g')  
  .attr("transform", `translate(0, ${chartHeight})`)  
  .call(d3.axisBottom(scaleX));
```

Definizione scaleY e asse Y

La funzione `scaleY` invece viene creata tramite la funzione **`d3.scaleLinear()`** che mapperà la popolazione, un valore numerico. In questo caso, il dominio andrà da **0** a il valore **massimo** del dataset. La funzione **`.nice()`** serve per **arrotondare** l'intervallo in modo più leggibile.

Creiamo poi l'asse Y definendo anche le **"ticks"** e le linee orizzontali tratteggiate che attraversano il grafico, in modo da renderlo più leggibile.

```
//Mappiamo la popolazione sull'asse y
const scaleY = d3.scaleLinear()
  .domain([0, d3.max(data, d => d['population'])])
  .range([chartHeight, 0])
  .nice();

//Asse y avrà i valori della popolazione e delle linee orizzontali
gContainer.append('g')
  .call(d3.axisLeft(scaleY)
    .ticks(null, ".2s") //formatta i numeri notazione scientifica
    .tickSize(-chartWidth) //crea linee orizzontali da sinistra
  )
  .call(g => g.selectAll('line')
    .attr('stroke', 'lightgrey') //cambia il colore delle linee
    .attr('stroke-dasharray', '10, 2') //le rende tratteggiate
  );
```

Calcolo posizioni e dimensioni

Definiamo ora le funzioni per calcolare la **posizione** e la **dimensione** delle barre. Per ogni barra calcolano:

- Dove metterla, quindi le coordinate;
- Quanto deve essere larga, la lunghezza fissa della **scaleBand**;
- Quanto deve essere alta, quindi in base al "value" della popolazione

```
//Funzioni per calcolare la dimensione e il posizionamento delle barre
//Dato un valore ci deve restituire le coordinate x, y,
//la larghezza della barra(fissa) e l'altezza della barra in base
//al valore della popolazione
const getX = d => scaleX(d['name']) Show usages new *
const getY = d => scaleY(d['population']) Show usages new *
const getWidth = d => scaleX.bandwidth() Show usages new *
const getHeight = d => chartHeight - scaleY(d['population']) Show usages
```

Eventi mouseover e mouseout

Creiamo dei comportamenti per il mouseover e il mouseout.

Quando passiamo con il mouse su una barra questa deve **cambiare colore** e deve **apparire quanto vale** quella barra.

Quando non abbiamo più il mouse su quella barra allora ritorna come era prima.

```
const onMouseover = (event, d) => { Show usages new *  
  d3.select(event.currentTarget)  
    .attr('fill', 'orange')  
    .append('title')  
    .text(d => d3.format(".3s")(d['population']))  
}  
  
const onMouseout = function (event, d) { Show usages new *  
  d3.select(this)  
    .attr('fill', 'steelblue')  
}
```


Aggiunta delle barre

Infine, disegniamo ogni barra. Impostiamo tutti gli attributi con le funzioni create prima e aggiungiamo gli eventi **"mouseover"** e **"mouseout"**.

```
// Creiamo le barre come rettangoli
gContainer.selectAll('rect')
  .data(data)
  .join('rect')
  .attr('x', getX)
  .attr('y', getY)
  .attr('width', getWidth)
  .attr('height', getHeight)
  .attr('fill', 'steelblue')
  .on('mouseover', onMouseover)
  .on('mouseout', onMouseout)
```

Risultato

